USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Milestone 4 Verification Report

UM-CYDOS-012

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-CYDOS-012	1.0 · 2026-08-14	**PASS** — all exit criteria met on hardware	Internal

1. Abstract

Milestone 4 delivers the bytecode interpreter: a register machine with 16 registers and fixed 4-byte instructions, a host-side assembler, and syscalls into kernel services.

This is the milestone where cyd-os acquires isolation. Everything before it was trusted code sharing one address space with nothing between components but convention. From here, application code runs under a bounds check on every memory access, and the measurements in §6 are the evidence that a malformed or hostile program stops itself rather than the kernel.

2. The decision: register-based

Recorded as pending in UM-CYDOS-007 §6 and settled here.

	Stack machine	Register machine (selected)
Instructions per unit of work	High — operands pushed and popped	Low — operands named directly
Encoding	Compact, often 1 byte	4 bytes fixed
Decode	Trivial	One byte plus three fields
Producer complexity	Lower — expression trees map directly	Higher — needs register allocation
Dispatch overhead per useful operation	Paid several times	Paid once

Dispatch is the inner loop of everything this kernel will ever run. A stack machine spends a large share of its instructions moving operands rather than computing with them, and each of those pays the full dispatch cost. A register machine executes the same program in materially fewer instructions, which more than repays a wider encoding.

The cost is a more demanding producer. That cost lands on the host, where there is no memory constraint and no consequence for being slow — see §4.

`sum(1..10)` compiles to **70 executed instructions** including the loop, syscalls, and a call/return.

3. Instruction set

35 opcodes, within the ~40 the roadmap allows. Fixed 4-byte encoding:

```
byte 0   opcode
byte 1   a      destination register, or the sole operand
byte 2   b      source register, or immediate low byte
byte 3   c      source register / offset, or immediate high byte
```

Group	Opcodes
Control	`HALT NOP MOV LDI LDIH`
Arithmetic / logic	`ADD SUB MUL DIV MOD AND OR XOR SHL SHR SAR NOT NEG ADDI`
Comparison	`SEQ SNE SLT SLTU SLE SLEU`
Control flow	`JMP BRZ BRNZ CALL RET`
Memory	`LDW LDB STW STB`
Services	`SYS`

Comparisons produce 0 or 1 into a register rather than setting flags, so branches need only test a register for zero. That removes a whole class of state — condition codes with their own save/restore obligations across a context switch — from a kernel that has already paid once for forgetting to save processor state (UM-CYDOS-009 §6).

3.1 Choices that close failure modes

- **Register indices are validated, not masked.** An index of 16 or more faults. Masking would turn a producer bug into wrong answers instead of a diagnosed stop.
- **Shift counts are masked to 5 bits.** A shift of 32 or more is undefined in C; leaving it undefined would let a program's behaviour depend on the host compiler, which is precisely what a VM exists to prevent.
- **Word accesses must be 4-byte aligned.** Faulting is deterministic; the alternative is an unaligned-access exception in kernel context.
- **Branch displacements count instructions**, so a branch cannot target a misaligned address.

3.2 Return addresses live in kernel memory

`CALL` pushes to a kernel-side stack of 32 entries in the `vm_t` structure, not into the arena.

A program therefore **cannot address its own return addresses**. No amount of wild pointer arithmetic within its arena can corrupt where a function returns to, because that data is not in the arena at all. The entire family of stack-smashing bugs is unexpressible rather than defended against.

The cost is a fixed call depth and no stack-allocated locals in the conventional sense. For a kernel whose isolation is entirely software-enforced, being unable to express the attack is worth more than the flexibility.

4. The producer

`tools/vasm.py` assembles a text source to a C header containing a byte array and label offsets. It supports labels, `.string`, `.byte`, `.word`, `.align`, `.space`, character literals, `@label` for addresses, and instruction-relative branch resolution.

It runs on the **host**. Nothing about a producer needs to live in 176 KB of DRAM, and keeping it off the device means the on-device side stays a pure interpreter with no parsing, no symbol table, and no attack surface from untrusted text. `build.ps1` assembles `tools/*.vasm` into `kernel/generated/` as a build product.

The demonstration program is 120 bytes and 6 labels.

5. Preemption boundary

`vm_run(vm, quantum)` executes at most `quantum` instructions and returns `VM_RUN_QUANTUM` if the budget runs out. State lives entirely in `vm_t`, so execution resumes exactly where it stopped.

This is the safe-boundary preemption of UM-CYDOS-001 §4.2: a task hosting a VM can be descheduled between any two bytecode instructions without the program being able to observe it. The interpreter never has to be reentrant, and the timer interrupt never has to understand bytecode.

A program that never terminates costs its quantum and nothing further — proved in §6.3 rather than asserted.

6. Results

```
vm says sum(1..10) = 55
[4a] program   : HALTED ok  insns=70 resumes=4 status=0 fault=none
[4b] faults    : bounds=ok div0=ok opcode=ok reg=ok ret=ok align=ok  PASS
[4c] runaway   : PASS  bounded at 500 insns, no fault, kernel alive
[4d] predicate: PASS  vm_in_bounds agrees with arena_contains over 35 cases
```

6.1 The program

Arithmetic, a counted loop, a bounds-checked store and reload through arena memory, a call and return, and four syscalls producing UART output. It ran to `HALT` with exit status 0 in 70 instructions.

It was run with a quantum of 16, so it was **suspended and resumed 4 times** mid-execution and produced the correct answer regardless — which is the property that matters, not the answer itself.

6.2 Containment

Six deliberately malformed programs, each hand-encoded as raw bytes rather than assembled. Routing them through the assembler would only have demonstrated that well-formed programs behave; the claim under test is about malformed ones.

Probe	Attempts	Expected fault	Result
`bounds`	Store to offset `0xFFF0` in a 2 KB arena	`VM_FAULT_BOUNDS`	ok
`div0`	Divide by zero	`VM_FAULT_DIV0`	ok
`opcode`	Byte `0xFF` as an instruction	`VM_FAULT_OPCODE`	ok
`reg`	`MOV r16, r0`	`VM_FAULT_REG`	ok
`ret`	`RET` with an empty call stack	`VM_FAULT_RET`	ok
`align`	Word load at offset 1	`VM_FAULT_ALIGN`	ok

Each stopped its own program with the correct code and returned control normally. The strongest evidence is indirect: every line of output after this test exists only because the kernel survived all six.

6.3 Runaway

A one-instruction program branching to itself. Executed exactly 500 instructions under a 500-instruction quantum, reported `VM_RUN_QUANTUM`, raised no fault, and left the kernel running. An unterminating program is a scheduling cost, not a hang.

6.4 The two bounds predicates agree

`vm_in_bounds()` duplicates `arena_contains()` so the interpreter's inner loop need not make a function call per access. Duplicated logic drifts, so the agreement is **tested rather than trusted**: 35 combinations of offset and length, including zero lengths, the exact end, one past the end, and lengths chosen to wrap the address space. All agree.

6.5 Regression

M3 self-test passing in full. M2 workload unchanged: 3,418 ticks, switches 1140/1139/1139, guards intact, `corrupt=0`.

7. `kstring.c`, and a trap avoided

The link failed on an undefined reference to `memcpy` from a function whose source never mentions it. GCC synthesises calls to `memcpy` / `memset` from ordinary C — byte-copy loops, struct assignments, large local initialisers — and does so even under `-fno-builtin`, which only governs the treatment of those names when written explicitly. Under `-nostdlib` nothing supplies them.

`kernel/kstring.c` provides `memcpy`, `memset`, `memmove` and `memcmp`.

The trap worth recording is what happens next: GCC will recognise the copy loop *inside* `memcpy` as a memcpy and rewrite it into a call to itself. That bug is silent, infinitely recursive, and would surface as a stack overflow in whatever unrelated code first copied a struct. The build now passes `-fno-tree-loop-distribute-patterns`, which is the supported way to prevent it.

8. Metrics

Quantity	Value
Opcodes	35 (roadmap ceiling ~40)
Instruction width	4 B fixed
Registers	16
Call depth	32, in kernel memory
Demo program	120 B, 6 labels
Instructions to compute `sum(1..10)`	70
Quantum resumptions during demo	4
Fault classes exercised	6 of 10 defined
Bounds predicate cases cross-checked	35
Image size	10,144 B

9. What M4 does not establish

- **No arena-relative program loading.** A program is copied to offset 0 and addresses everything relative to its arena. There is no relocation, no linking of separately assembled units, and no code/data separation within the arena — a program can overwrite its own instructions.
- **Only 6 of 10 fault classes are exercised.** `VM_FAULT_PC`, `VM_FAULT_CALL_DEPTH`, `VM_FAULT_SYSCALL` and `VM_FAULT_STRING` are implemented but untested on hardware.
- **No performance measurement.** Dispatch cost per instruction is unmeasured. The `switch` was chosen over a computed-goto table for diagnosability, and that trade should be revisited only against a measurement.
- **No VM task integration.** The VM runs from `kmain`'s self-test, not from a native task under the scheduler. Wiring `vm_run()` into a task's loop is the obvious next step and is not done.
- **No multiple concurrent VMs.** One at a time, though nothing in the design prevents more — `vm_t` carries all state.
- **Syscalls are unaudited for argument validation** beyond `PUTS`, which walks its string one bounds-checked byte at a time and faults on a missing terminator rather than reading past the end.

10. References

- UM-CYDOS-001 §4.2 — isolation model and safe-boundary preemption
- UM-CYDOS-007 §6 — M4 deliverable and the ISA decision recorded as pending
- UM-CYDOS-010 §5.2 — `arena_contains()` and the offset-domain argument
- UM-CYDOS-009 §6 — the saved-state failure that motivated §3 flag-free design
- `kernel/vm.h` — ISA definition and isolation rationale

- `kernel/vm.c` — dispatch loop and checked accessors
- `tools/vasm.py` — the producer